

QEMU Internals

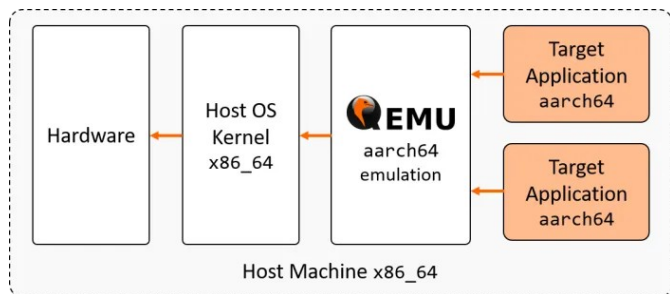
Agenda

- ❑ **Introduction**
- ❑ **TCG Translator**
- ❑ **QEMU Source Code**
- ❑ **QEMU Device Model**
- ❑ **PC Hardware Initialization**
- ❑ **Virtual Memory (SoftMMU)**
- ❑ **QEMU Architecture**

Introduction

What is QEMU?

- QEMU stands for Quick Emulator, is a machine emulator using dynamic binary translation.
- It can emulate several CPUs, devices and buses.
- ex. emulate ARM machines on x86 machine.
That tends to translation from **ARM's ISA** → **x86 ISA**.
- So, Qemu translates from
Target CPU instruction → Host CPU instruction (runtime).



Emulated IO

- Input
- Display
- Character / Serial
- Audio
- Block
- Networking

Bus emulation

- MMIO
- PCI / PCIe
- SSI (SPI)
- I2C
- PCMCIA
- SCSI
- USB

Alpha
Arm (arm, aarch64)
AVR
Cris
Hexagon
PA-RISC (hppa)
x86 (i386, x86_64)
Loongarch
m68k
Microblaze
MIPS (mips*)
OpenRISC

Introduction

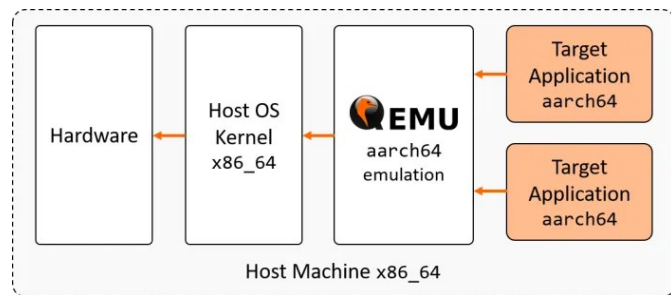
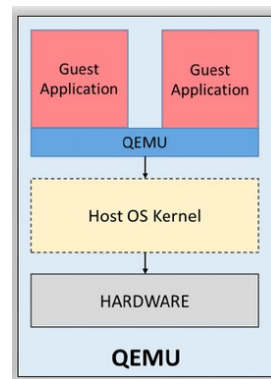
QEMU Modes

- QEMU has two operating modes
 - **Full system emulation**
QEMU emulates the full system including the cpu and various devices.

ex. Run ARM Linux board (Beaglebone black, raspberry pi,...) inside QEMU program you have at x86 Linux machine.

ex. Run an OS like Windows (unmodified) machine inside QEMU program you have at x86 Linux machine.
 - **User mode emulation**
QEMU can launch processes compiled for one CPU on another CPU.

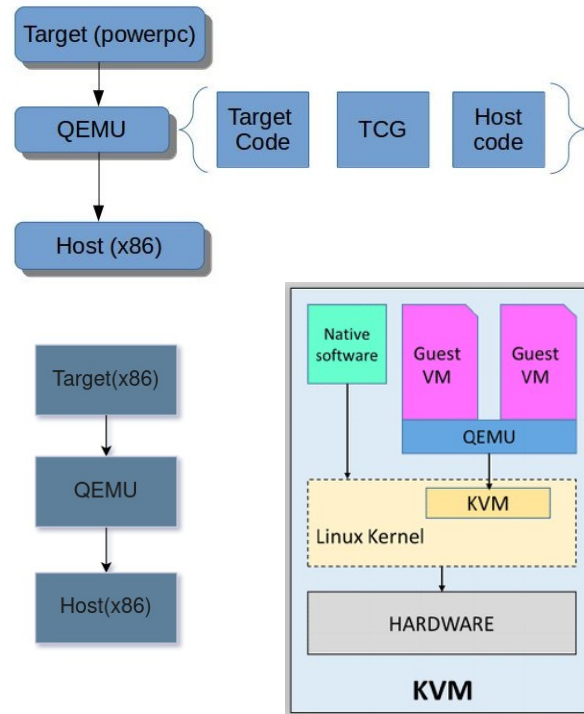
ex. Write a program compiled for ARM and run it inside your x86 Linux.



Introduction

QEMU & KVM

- QEMU uses **dynamic binary translation** to translate from Target CPU instruction → Host CPU instruction at runtime.
- The binary translator that does this job is known as **Tiny Code Generator (TCG)**.
- TCG uses dynamic binary translation also known as Just in Time (**JIT**) compilation to emulate the guest.
- There are cases where the source and the target architecture are the same (x86 on x86).
- At that time, you can make things faster unchanged code directly on the CPU **without translating** it.
- This is where **KVM** comes, it works as an **accelerator** for QEMU as it can execute CPU instructions natively without the need of a **JIT** compilation.
- Instead of calling **TCG** to execute guest code, it talks to the `/dev/kvm` device for CPU virtualization and QEMU handles the rest of hardware emulation.



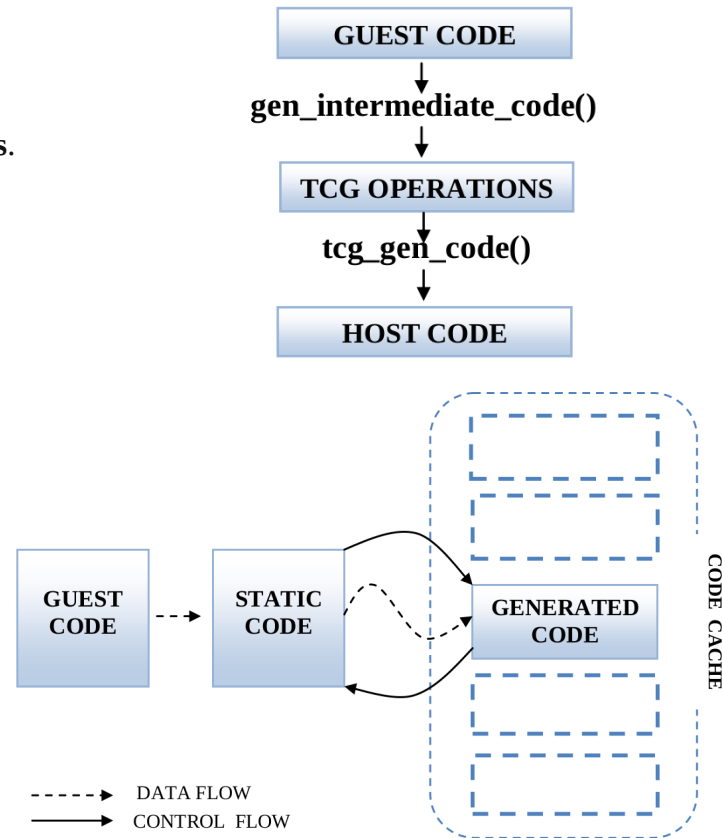
```
static const TypeInfo kvm_accel_type = {
    .name = TYPE_KVM_ACCEL,
    .parent = TYPE_ACCEL,
    .instance_init = kvm_accel_instance_init,
    .class_init = kvm_accel_class_init,
    .instance_size = sizeof(KVMState),
};

static const TypeInfo xen_accel_type = {
    .name = TYPE_XEN_ACCEL,
    .parent = TYPE_ACCEL,
    .class_init = xen_accel_class_init,
};
```

TCG Translator

Dynamic Translation

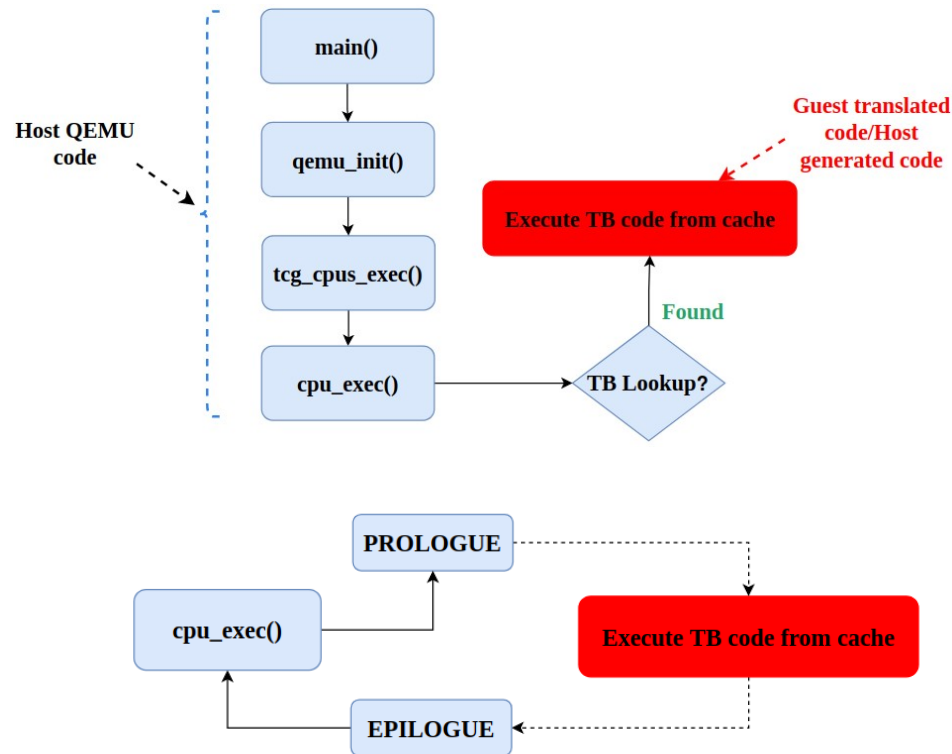
- The translation task consists of two parts
 - Each block of target code called **translation block (TB)**, is converted into **TCG ops**. **TCG ops** is machine-independent intermediate notation.
 - **TCG ops** for the TB is converted to Host code for the host's architecture by **TCG**.
- The dynamic translation converts code only when needed such that, spending the maximum time executing the **generated code** than executing the code generation.
- Every time code is generated from the TB it is stored in the **code cache** before being executed.
- Most of the time the same TBs are required again and again, owing to what is called **Locality Reference**, so instead of re-generating the same code it is best to save it.
- Once the code cache is full, to make things simple, the entire **code cache** is **flushed**.



TCG Translator

Dynamic Translation

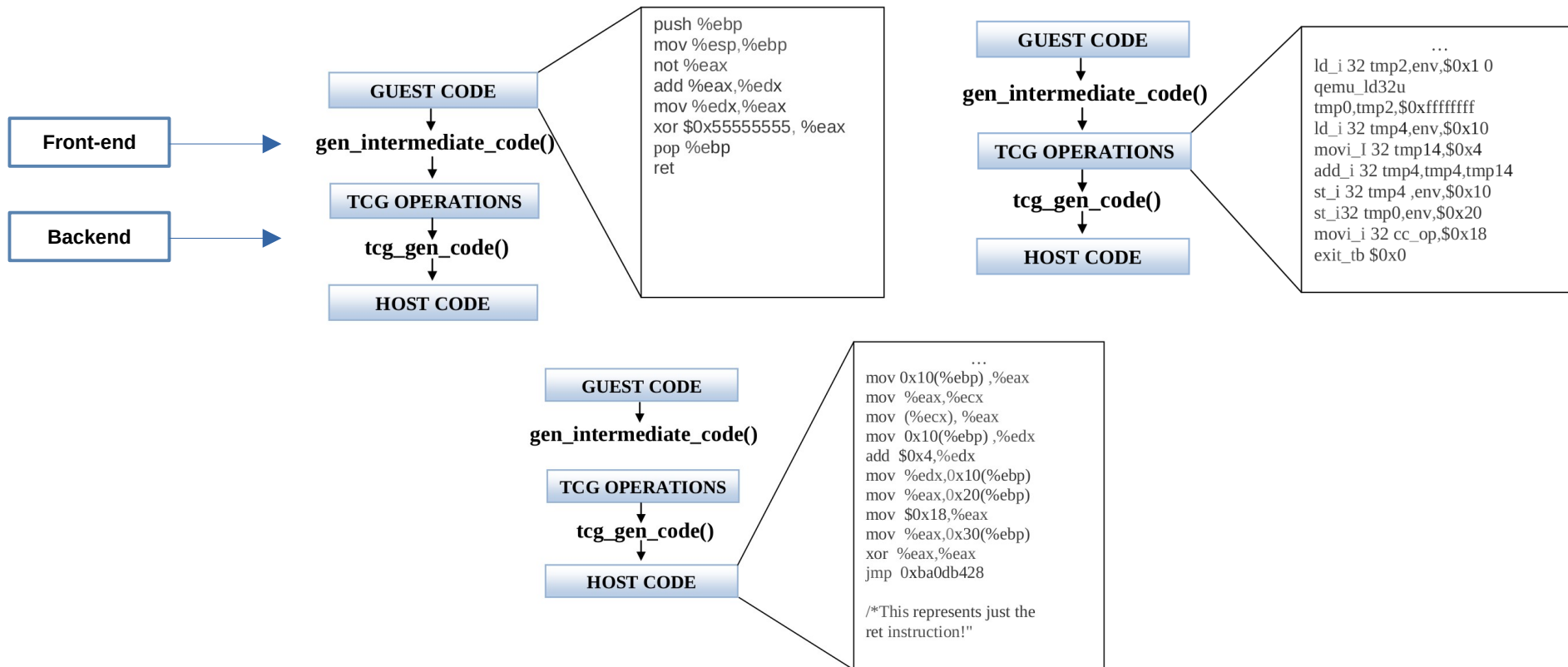
- Before the control flow goes to execute the generated code (Target/Guest), QEMU add and execute pre-generated special assembly code before a guest function is called **Function Prologue and Epilogue**.
- **Function Prologue**
 - Pushes the current **base pointer** onto the stack, such that it can be restored later.
 - Replaces the old **base pointer** with the current stack pointer such that a new stack will be created on top of the old stack.
 - Moves the **stack pointer** further along the stack to make room in the current stack frame for the function's local variables.
- **Function Epilogue**
 - Replaces the stack pointer with the current base pointer, so the stack pointer is restored to its value before the prologue.
 - Pops the base pointer off the stack, so it is restored to its value before the prologue.
 - Returns to the calling function, by popping the previous frame's program counter off the stack and jumping to it.
- For better understanding [Stack frame layout on x86-64](#).



TCG Translator

Dynamic Translation

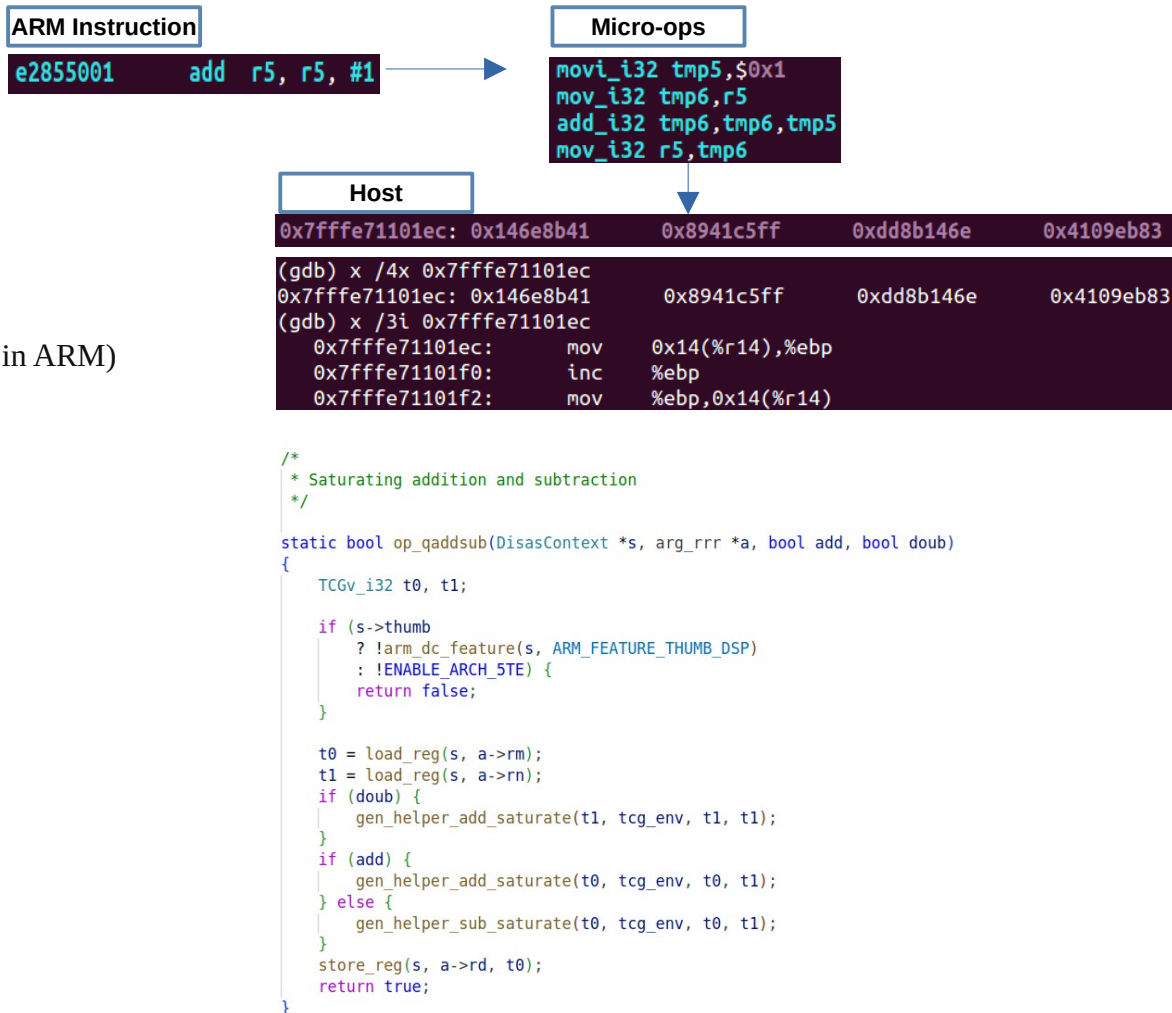
- Generated host Code shown in assembly for readability.



TCG Translator

Dynamic Translation

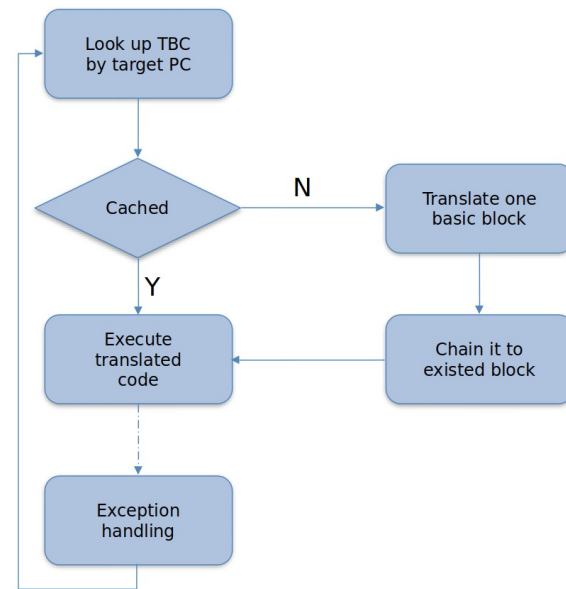
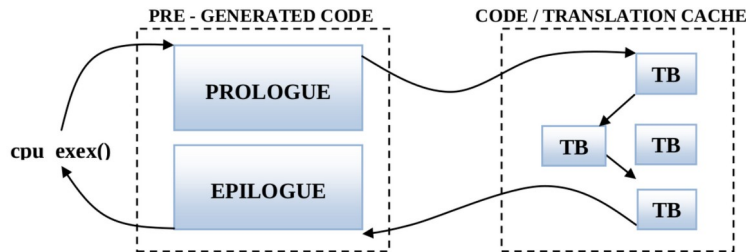
- Front-end Components
 - Micro-ops
 - Temporary variables
 - Helper functions (Complicated instructions ex. QADD in ARM)
QDADD : adds two values in **Rm** and **Rn** registers and saturating the result to the signed range $-2^{31} \leq x \leq 2^{31} - 1$.
Rm, Rn : are the registers holding the operands.
- Backend Components
 - Host code transformation.
- ARM ex.



TCG Translator

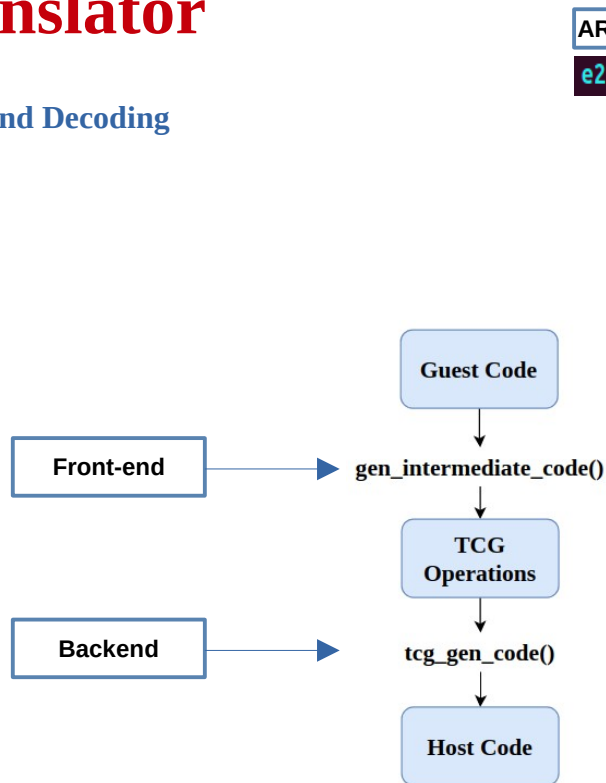
Chaining TBs

- Because returning from the code cache to the static code (QEMU code) and jumping back into the code cache is generally slow, QEMU chains every TB to the next TB.
- After the execution of one TB, the execution directly jumps to the next TB without returning to the static code.



TCG Translator

Front-end/backend Decoding



ARM Instruction

e2855001 add r5, r5, #1

Micro-ops

```
movi_i32 tmp5,$0x1
mov_i32  tmp6,r5
add_i32  tmp6,tmp6,tmp5
mov_i32  r5,tmp6
```

Host

```
0x7fffe71101ec: 0x146e8b41    0x8941c5ff    0xdd8b146e    0x4109eb83
(gdb) x /4x 0x7fffe71101ec
0x7fffe71101ec: 0x146e8b41    0x8941c5ff    0xdd8b146e    0x4109eb83
(gdb) x /3i 0x7fffe71101ec
0x7fffe71101ec:    mov    0x14(%r14),%ebp
0x7fffe71101f0:    inc    %ebp
0x7fffe71101f2:    mov    %ebp,0x14(%r14)
```

TCG Translator

Front-end Decoding

- Qemu defines **instruction patterns** to describe client cpu instructions.
- Each architecture has its own files which describing the its instruction patterns.
ex. ARM : /target/arm/tcg/**a32.decode**
- While compilation
/scripts/**decodetree.py** script generates the declaration only of the corresponding instruction description and store it in output file after compilation
ex. ARM : /build/libqemu-arm-softmmu.fa.p/**decode-insn32.c.inc**
- Afterwords, we should implement the declared functions
ex. ARM : /target/arm/tcg/**translate.c**
- While qemu is running, it parse the binary instructions (insn, arguments).

translate.c

```
static bool trans_ADR(DisasContext *s, arg_ri *a)
{
    store_reg_bx(s, a->rd, add_reg_for_lit(s, 15, a->imm));
    return true;
}

static bool trans_MOVW(DisasContext *s, arg_MOVW *a)
{
    if (!ENABLE_ARCH_6T2) {
        return false;
    }

    store_reg(s, a->rd, tcg_constant_i32(a->imm));
    return true;
}
```

decode-insn32.c.inc

```
typedef struct {
    int rd;
    int imm;
} arg_ri;

typedef struct {
    int rd;
    int imm;
} arg_MOVW;

static bool trans_ADR(DisasContext *s, arg_ri *a);
static bool trans_MOVW(DisasContext *s, arg_MOVW *a);
```

ARM Instruction

e2855001 add r5, r5, #1

Micro-ops

movi_i32 tmp5,\$0x1
mov_i32 tmp6,r5
add_i32 tmp6,tmp6,tmp5
mov_i32 r5,tmp6



TCG Translator

Front-end Decoding

- Decode Tree
 - /scripts/**decodetree.py** parses the input file **a32.decode**, and understands all patterns and get extract **fixedbits**, **fixedmask**.
 - The combination of **fixedbits** and **fixedmask** describes the condition under which the pattern is matched.
 - In support of patterns, and in addition to **insn pattern**, one may declare **fields**, **argument sets**, and **formats**, each of which may be re-used to simplify further definitions.
 - **Field**
 - ex. **%disp 0:s16** this means there is a new field called **disp** will be generated as follows,
 - extract from the instruction **i**, the bits starts from **0** and length **16**.
 - ex. **%disp12 0:s1 1:1 2:10** indicates that
 - the field is composed of **three** parts, so the generated code includes the corresponding **shift** and **OR** operations.
 - ex. **%shimm8 5:s8 13:1 !function=expand_shimm8** indicates that
 - after extracting the field from the instruction **i**, pass it to the function called **expand_shimm8**.



```
a32.decode x tcg.c qemu-config.c decode-insns.c.inc qdev.c
target > arm > tcg > a32.decode
79 %imm16 16:4 0:12
80 @mov16 ---- ... rd:4 ..... &ri imm=%im
81
82 MOVW .... 0011 0000 .... @mov16
83 MOVT .... 0011 0100 .... @mov16
84
85 # Data-processing (register-shifted register)
86
87 @s_rrr_shr ---- ... s:1 rn:4 rd:4 rs:4 . shty:2 . rm:4 \
88 | | | | | &s_rrr_shr
89 @s_rxr_shr ---- ... s:1 .... rd:4 rs:4 . shty:2 . rm:4 \
90 | | | | | &s_rrr_shr rn=0
91 @s_xrr_shr ---- ... . rn:4 .... rs:4 . shty:2 . rm:4 \
92 | | | | | &s_rrr_shr rd=0 s=1
```

```
(insn & fixedmask) == fixedbits
```

Input	Generated code
%disp 0:s16	sextract(i, 0, 16)
%imm9 16:6 10:3	extract(i, 16, 6) << 3 extract(i, 10, 3)
%disp12 0:s1 1:1 2:10	sextract(i, 0, 1) << 11 extract(i, 1, 1) << 10 extract(i, 2, 10)
%shimm8 5:s8 13:1 !function=expand_shimm8	expand_shimm8(sextract(i, 5, 8) << 1 extract(i, 13, 1))

TCG Translator

Front-end Decoding

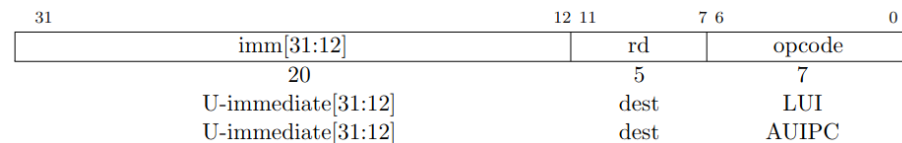
- Decode Tree
 - **Argument set**
ex. **&u imm rd** this means **u** is a set (**C struct**) of arguments consists of **imm** and **rd**.
 - **Format**
ex. **@u &u imm=%imm_u %rd**
this means there is a new format definition called **u**.
each **.** represents 1 bit that will be covered with a **field** or a final 0 or 1 from the **Instruction Pattern**.

So, **@u** format consists of unspecified **20bits** placeholder that can take any value then **5bits** the same, then **7bits** the same.
and takes **&u** an argument set (imm, rd) that will be extracted as fields **%imm** and **%rd**.

➤ Instruction Pattern

```
lui ..... 0110111 @u
@u ..... &u imm=%imm_u %rd
```

```
%imm_u 12:s20 !function=ex_shift_12
%rd 7:5
&u imm rd
@u ..... &u imm=%imm_u %rd
```



```
lui ..... 0110111 @u
# Argument sets:
&u imm rd
# Formats 32:
@u ..... &u imm=%imm_u %rd
# Fields:
%rd 7:5
# immediates:
%imm_u 12:s20 !function=ex_shift_12
```

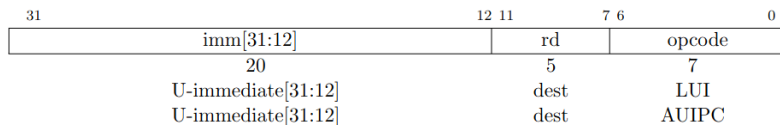
TCG Translator

Front-end Decoding

- Example
 - **lui** (riscv processor load upper immediate)
trans_lui() should be implemented to generate **micro-ops** then pass it to backend translator.
 - **trans_XXX** : where **XXX** is the instruction name.
 - **tcg_gen_op2()** for all operations having 2 operands.
 - At the end, **tcg_emi_op()** allocates through **tcg_op_alloc()** the operation according to **opc**, **nargs** then insert it in a linked list contains the ongoing operations **QTAIL_INSERT_TAIL()**.

lui rd, imm → Target Code

LUI places the U-immediate value in the top 20 bits of the destination register rd, filling in the lowest 12 bits with zeros.



mov_i32 rd, imm

Micro-ops

```
typedef struct {
    int rd;
    int imm;
} arg_lui;

static bool trans_lui(DisasContext *ctx, arg_lui *a);
```

decode-insn32.c.inc

```
C trans_rvi.c.inc 1 X  decodetree.py  C qemu-option.c  insn32
target > riscv > insn_trans > C trans_rvi.c.inc > trans_auiipc(DisasContext *, arg_lui *a)
33 static bool trans_lui(DisasContext *ctx, arg_lui *a)
34 {
35     gen_set_gpri(ctx, a->rd, a->imm);
36     return true;
37 }
```

```
static void gen_set_gpri(DisasContext *ctx, int reg_num, target_long imm)
{
    tcg_gen_movi_tl(cpu_gpr[reg_num], (int32_t)imm);
}
```

```
void NI tcg_gen_op2(TCGOpcode opc, TCGArg a1, TCGArg a2)
{
    TCGOp *op = tcg_emit_op(opc, 2);
    op->args[0] = a1;
    op->args[1] = a2;
}
```

```
TCGOp *tcg_emit_op(TCGOpcode opc, unsigned nargs)
{
    TCGOp *op = tcg_op_alloc(opc, nargs);

    if (tcg_ctx->emit_before_op) {
        QTAILQ_INSERT_BEFORE(tcg_ctx->emit_before_op, op, link);
    } else {
        QTAILQ_INSERT_TAIL(&tcg_ctx->ops, op, link);
    }

    return op;
}
```

TCG Translator

Backend Decoding

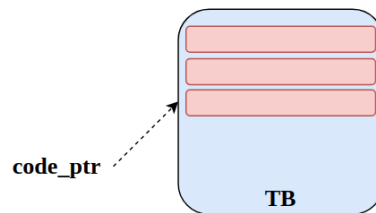
- TCG front-end decodes and maps the client CPU registers to QEMU virtual registers (**rd Micro-ops** in this example).
- TCG backend generates the host CPU instructions and allocates the host CPU registers and maps the QEMU virtual registers (**rd Micro-ops** in this example) to the physical registers of the host CPU.
- tcg_gen_code()** : starts generating the host TB code for the intermediate micro-ops.
- tcg_reg_alloc_op()** : default generic operation allocating host cpu operation.
- tcg_reg_alloc_mov()** : for all move operations.

```
static void tcg_out_dat_reg(TCGContext *s, ARMCond cond, ARMInsn opc,
                           TCGReg rd, TCGReg rn, TCGReg rm, int shift)
{
    tcg_out32(s, (cond << 28) | (0 << 25) | opc |
                (rn << 16) | (rd << 12) | shift | rm);
}

static void tcg_out_mov_reg(TCGContext *s, ARMCond cond, TCGReg rd, TCGReg rm)
{
    /* Simple reg-reg move, optimising out the 'do nothing' case */
    if (rd != rm) {
        tcg_out_dat_reg(s, cond, ARITH_MOV, rd, 0, rm, SHIFT_IMM_LSL(0));
    }
}

static __attribute__((unused)) inline void tcg_out32(TCGContext *s, uint32_t v)
{
    if (TCG_TARGET_INSN_UNIT_SIZE == 4) {
        *s->code_ptr++ = v;
    } else {
        tcg_insn_unit *p = s->code_ptr;
        memcpy(p, &v, sizeof(v));
        s->code_ptr = p + (4 / TCG_TARGET_INSN_UNIT_SIZE);
    }
}
```

arm op-code
1101



riscv Target

lui rd, imm

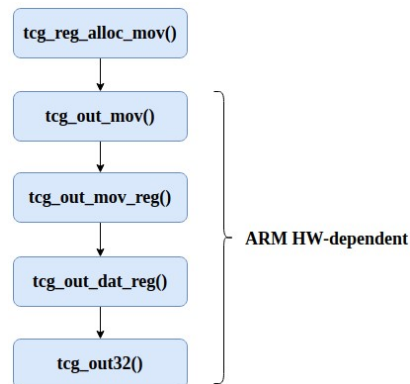
Micro-ops

mov_i32 rd, imm

arm Host

mov r2, #imm_val

```
int tcg_gen_code(TCGContext *s, TranslationBlock *tb, target_ulong pc_start)
{
    QTAILQ_FOREACH(op, &s->ops, link) {
        TCGOpcode opc = op->opc;
        ...
        switch (opc) {
            case INDEX_op_mov_i32:
            case INDEX_op_mov_i64:
            case INDEX_op_mov_vec:
                tcg_reg_alloc_mov(s, op);
            ...
            default:
                tcg_reg_alloc_op(s, op);
        }
    }
}
```



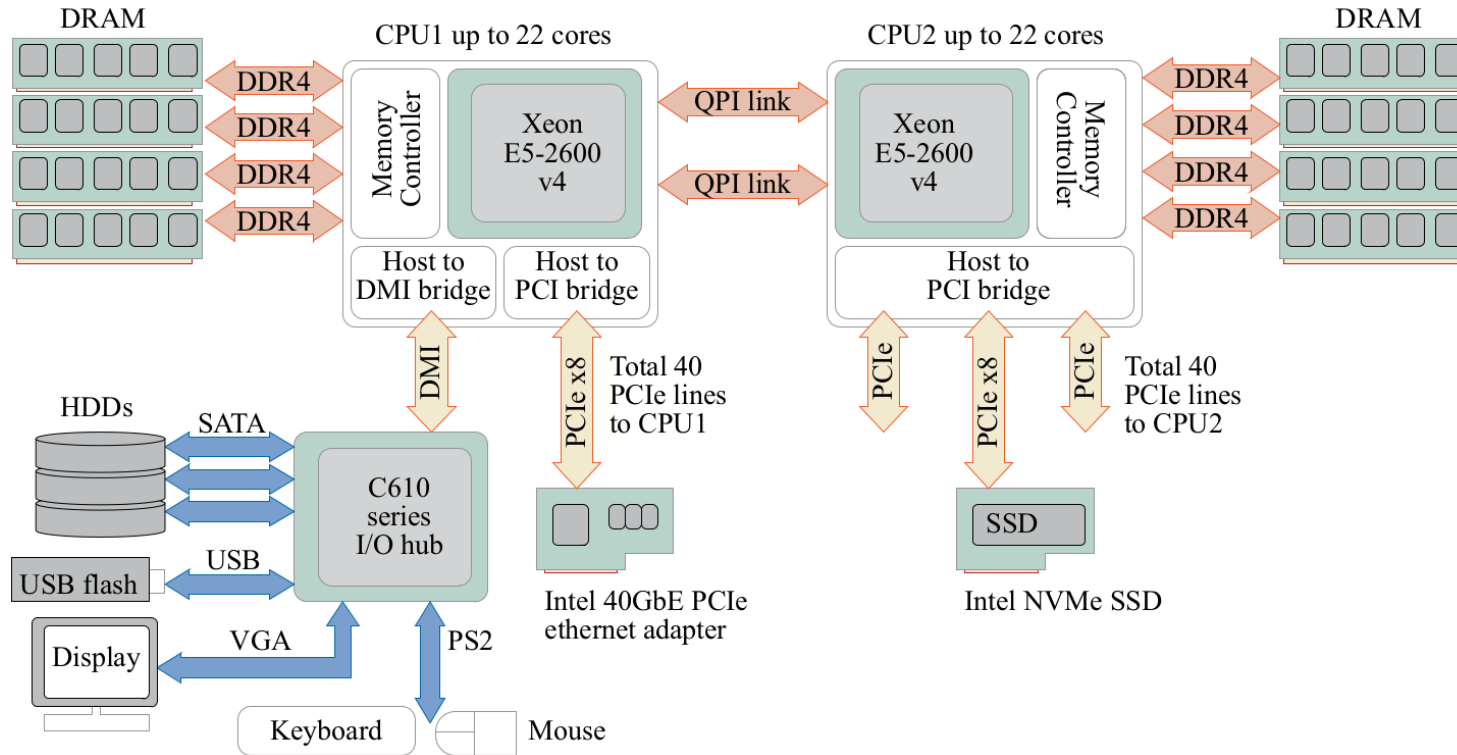
QEMU Source Code

Important Directories

Directory/File	Description
hw/	Contains all the HW emulations (CPUs, Buses, Boards, I/O devices).
target/	Contains all the HW targets decoding and translations.
util/qemu-option.c util/qemu-config.c	Contains command line parsing the options and configurations.
hw/core/qdev.c	Device model codes.
monitor/ monitor/qmp.c	Used to interact with a running QEMU instance (attach new drives, save snapshot,...). Contains the QEMU Machine Protocol (QMP), which is a json based protocol that allows applications such as libvirt to communicate with a running QEMU instance
qom/	Contains QEMU Object Model codes (QOM).
softmmu/main.c	Contains qemu entry point main(), initializations and starts emulating the specified HW machine.
hw/mem/	Manages virtual memory (SoftMMU) subsystem.
backend/hostmem.c	Manages and allocates host memories.
util/main-loop.c	Manages main event loop qemu architecture.

QEMU Device Model

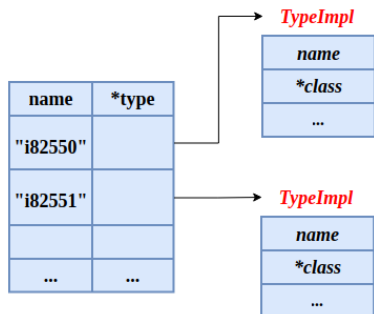
Physical Devices



QEMU Device Model

Device Type Registration

- Each device simulated should have a corresponding device type **TypeInfo** that's created by the user to finally instantiate a new device.
- A new device can be registered using **type_register(TypeInfo)** users API with new **name** and referring to the **parent** name.
- type_register()** generates a **TypeImpl** corresponding to TypeInfo by calling **type_new()**.
- Then adds **TypeImpl** to internal hash table called **type_table** with name as the keyword.
- TypeImpl** is very similar to TypeInfo, however TypeInfo saves static registration data, while TypeImpl saves dynamic operation data.



```
struct TypeImpl
{
    const char *name;

    size_t class_size;

    size_t instance_size;
    size_t instance_align;

    void (*class_init)(ObjectClass *klass, void *data);
    void (*class_base_init)(ObjectClass *klass, void *data);

    void *class_data;

    void (*instance_init)(Object *obj);
    void (*instance_post_init)(Object *obj);
    void (*instance_finalize)(Object *obj);

    bool abstract;

    const char *parent;
    TypeImpl *parent_type;

    ObjectClass *class;

    int num_interfaces;
    InterfaceImpl interfaces[MAX_INTERFACES];
};
```

```
static TypeImpl *type_register_internal(const TypeInfo *info)
{
    TypeImpl *ti;
    ti = type_new(info);
    type_table_add(ti);
    return ti;
}

TypeImpl *type_register(const TypeInfo *info)
{
    assert(info->parent);
    return type_register_internal(info);
}
```

```
struct TypeInfo
{
    const char *name;
    const char *parent;

    size_t instance_size;
    size_t instance_align;
    void (*instance_init)(Object *obj);
    void (*instance_post_init)(Object *obj);
    void (*instance_finalize)(Object *obj);

    bool abstract;
    size_t class_size;

    void (*class_init)(ObjectClass *klass, void *data);
    void (*class_base_init)(ObjectClass *klass, void *data);
    void *class_data;

    InterfaceInfo *interfaces;
};

static void eepr0100_register_types(void)
{
    size_t i;
    for (i = 0; i < ARRAY_SIZE(eepr0100_devices); i++) {
        TypeInfo type_info = {};
        E100PCIDeviceInfo *info = &eepr0100_devices[i];

        type_info.name = info->name;
        type_info.parent = TYPE_PCI_DEVICE;
        type_info.class_init = eepr0100_class_init;
        type_info.instance_size = sizeof(E100PCIDeviceInfo);
        type_info.instance_init = eepr0100_instance_init;
        type_info.interfaces = (InterfaceInfo[]) {
            { INTERFACE_CONVENTIONAL_PCI_DEVICE },
            {}
        };

        type_register(&type_info);
    }
}

type_init(eepr0100_register_types)
```

QEMU Device Model

Device Type Registration

- **type_init()** is actually a **module_init()** wrapper.
- After pre-processing,
static void do_qemu_init_eepro100_register_types(void) {}
- This **__attribute__((constructor))** compiler directive is to make sure module initialization is called at early stage (startup code) before QEMU **main()** program.

```
#define type_init(function) module_init(function, MODULE_INIT_QOM)
#define module_init(function, type) \
static void __attribute__((constructor)) do_qemu_init_## function(void) \
{ \
    register_module_init(function, type); \
}

static void eepro100_register_types(void)
{
    size_t i;
    for (i = 0; i < ARRAY_SIZE(e100_devices); i++) {
        TypeInfo type_info = {};
        E100PCIDeviceInfo *info = &e100_devices[i];

        type_info.name = info->name;
        type_info.parent = TYPE_PCI_DEVICE;
        type_info.class_init = eepro100_class_init;
        type_info.instance_size = sizeof(EPRO100State);
        type_info.instance_init = eepro100_instance_init;
        type_info.interfaces = (InterfaceInfo[]) {
            { INTERFACE_CONVENTIONAL_PCI_DEVICE },
            { },
        };

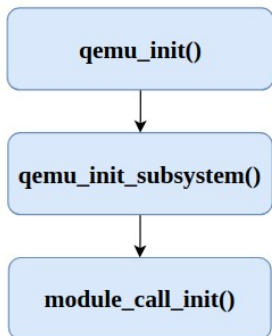
        type_register(&type_info);
    }
}

type_init(eepro100_register_types)
```

QEMU Device Model

Device Type Registration

- QEMU maintains a global linked list array called **init_type_list**, which points to different types of registered functions.
- register_module_init()** inserts a new **ModuleEntry** for **eeepro100_register_types**.
- QEMU Initialization
`qemu_init()` → `qemu_init_subsystems()` →
`module_call_init(MODULE_INIT_QOM)`
- module_call_init(MODULE_INIT_QOM)** executes the init member functions in the linked list items one by one.



```

void register_module_init(void (*fn)(void), module_init_type type)
{
    ModuleEntry *e;
    ModuleTypeList *l;

    e = g_malloc0(sizeof(*e));
    e->init = fn;
    e->type = type;

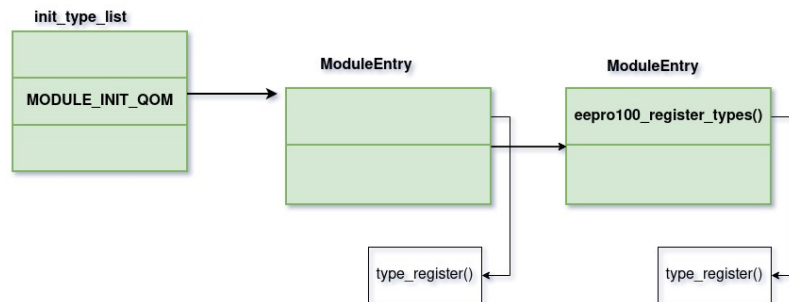
    l = find_type(type);

    QTAILQ_INSERT_TAIL(l, e, node);
}

#define QTAILQ_INSERT_TAIL(head, elm, field) do {
    (elm)->field.tqe_next = NULL;
    (elm)->field.tqe_circ.tql_prev = (head)->field.tqe_circ.tql_prev;
    (head)->field.tqe_circ.tql_prev->tqe_next = (elm);
    (head)->field.tqe_circ.tql_prev = (elm)->field.tqe_circ;
} while (/*CONSTCOND*/0)
  
```

```

#define type_init(function) module_init(function, MODULE_INIT_QOM)
#define module_init(function, type)
static void __attribute__((constructor)) do_qemu_init_## function(void)
{
    register_module_init(function, type);
}
  
```



```

static void eeepro100_register_types(void)
{
    size_t i;
    for (i = 0; i < ARRAY_SIZE(e100_devices); i++) {
        TypeInfo type_info = {};
        E100PCIDeviceInfo *info = &e100_devices[i];

        type_info.name = info->name;
        type_info.parent = TYPE_PCI_DEVICE;
        type_info.class_init = eeepro100_class_init;
        type_info.instance_size = sizeof(E100PCIDeviceState);
        type_info.instance_init = eeepro100_instance_init;
        type_info.interfaces = (InterfaceInfo[]) {
            { INTERFACE_CONVENTIONAL_PCI_DEVICE },
            { },
        };

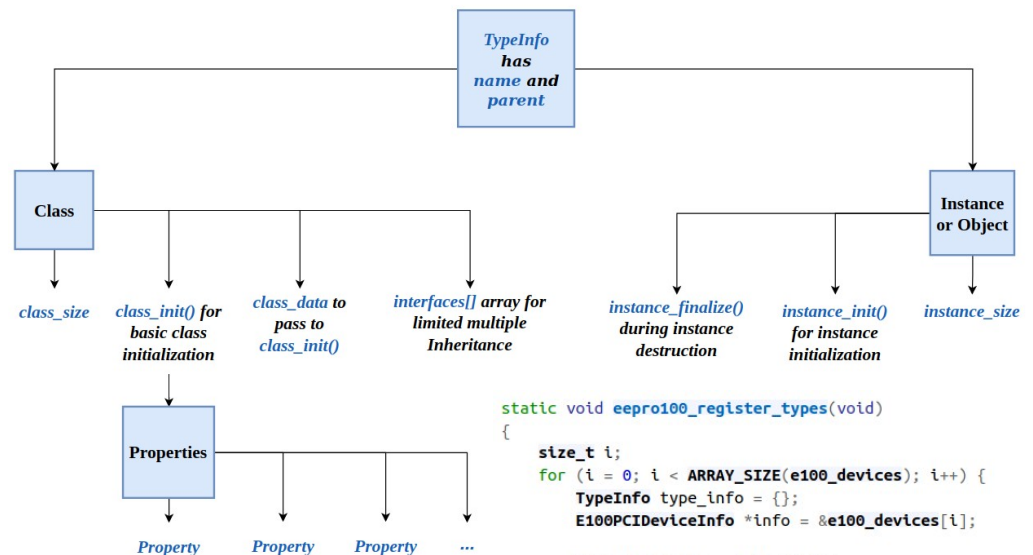
        type_register(&type_info);
    }
}

type_init(eeepro100_register_types)
  
```

QEMU Device Model

QEMU Object Model

QEMU User Perspective



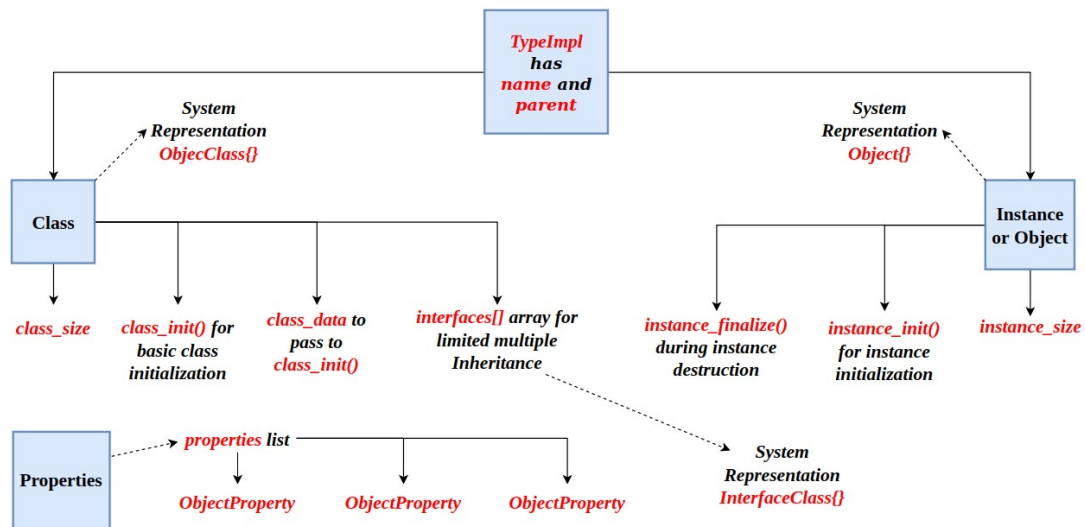
```
static void eepr0100_register_types(void)
{
    size_t i;
    for (i = 0; i < ARRAY_SIZE(e100_devices); i++) {
        TypeInfo type_info = {};
        E100PCIDeviceInfo *info = &e100_devices[i];

        type_info.name = info->name;
        type_info.parent = TYPE_PCI_DEVICE;
        type_info.class_init = eepr0100_class_init;
        type_info.instance_size = sizeof(E100PCIDeviceInfo);
        type_info.instance_init = eepr0100_instance_init;
        type_info.interfaces = (InterfaceInfo[]) {
            { INTERFACE_CONVENTIONAL_PCI_DEVICE },
            { },
        };

        type_register(&type_info);
    }
}

type_init(eepr0100_register_types)
```

QEMU Internals Perspective



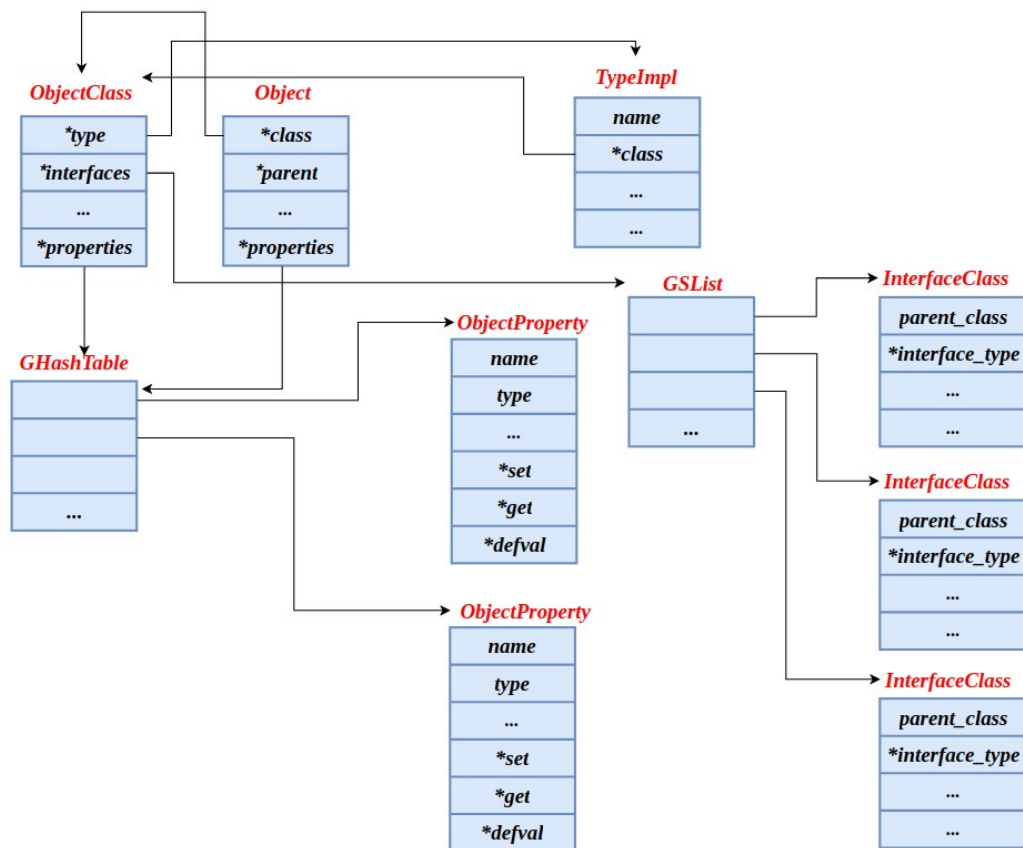
```
static const TypeInfo pci_device_type_info = {
    .name = TYPE_PCI_DEVICE,
    .parent = TYPE_DEVICE,
    .instance_size = sizeof(PCIDevice),
    .abstract = true,
    .class_size = sizeof(PCIDeviceClass),
    .class_init = pci_device_class_init,
    .class_base_init = pci_device_class_base_init,
};
```

```
struct A {
    struct B;
};
```

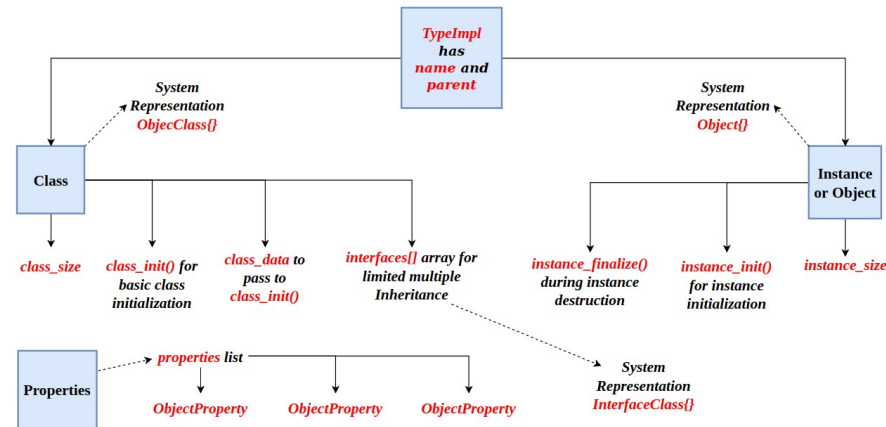
QEMU Device Model

QEMU Object Model

Internal Structures



QEMU Internals Perspective



QEMU Device Model

QEMU Object Model

- **TypeImpl** is the internal system representation of the created type.
- **ObjecClass** will be the base class of all classes.
- **Object** will be the base object for all instances.
- One **ObjectClass** can correspond to multiple Objects, while one **Object** only points to one **ObjectClass**.
- **InterfaceClass** will be the base class for all interfaces.

But what is actually an interface?

Interfaces are similar to normal types except that they are only defined by their classes and never carry any state.

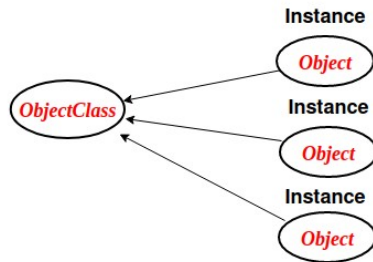
How new interface is created and registered by the user?

Like any other TypeInfo!.

And what the internal system representation

***InterfaceClass** consists of? It basically inherits from **ObjectClass**.*

- **ObjectProperty** is the system representation of any **Property** created by the user.



```
struct DeviceClass {  
    /* private: */  
    ObjectClass parent_class;  
    /* public: */  
};  
  
struct DeviceState {  
    /* private: */  
    Object parent_obj;  
    /* public: */  
};
```

```
struct BusClass {  
    ObjectClass parent_class;  
};  
  
struct BusState {  
    /* private: */  
    Object obj;  
    /* public: */  
};
```

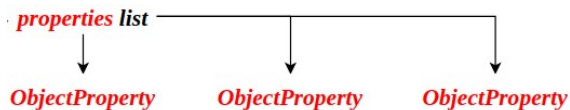
```
struct TypeImpl  
{  
};  
  
struct InterfaceClass  
{  
    ObjectClass parent_class;  
    /* private: */  
    ObjectClass *concrete_class;  
    Type interface_type;  
};
```

```
static const TypeInfo conventional_pci_interface_info = {  
    .name      = INTERFACE_CONVENTIONAL_PCI_DEVICE,  
    .parent    = TYPE_INTERFACE,  
};  
  
static void pci_register_types(void)  
{  
    type_register_static(&pci_bus_info);  
    type_register_static(&pcie_bus_info);  
    type_register_static(&cxl_bus_info);  
    type_register_static(&conventional_pci_interface_info);  
    type_register_static(&cxl_interface_info);  
    type_register_static(&pcie_interface_info);  
    type_register_static(&pci_device_type_info);  
}  
  
type_init(pci_register_types)
```

QEMU Device Model

QEMU Object Model

- Let's dissect the PCI device example



```
static const TypeInfo conventional_pci_interface_info = {
    .name      = INTERFACE_CONVENTIONAL_PCI_DEVICE,
    .parent    = TYPE_INTERFACE,
};

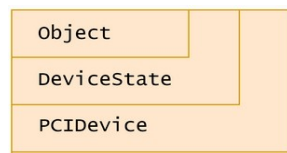
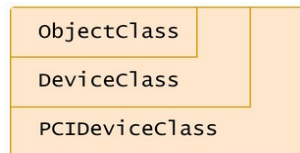
static void pci_device_class_init(ObjectClass *klass, void *data)
{
    DeviceClass *k = DEVICE_CLASS(klass);

    k->realize = pci_qdev_realize;
    k->unrealize = pci_qdev_unrealize;
    k->bus_type = TYPE_PCI_BUS;
    device_class_set_props(k, pci_props);
}
```

```
static const TypeInfo pci_device_type_info = {
    .name = TYPE_PCI_DEVICE,
    .parent = TYPE_DEVICE,
    .instance_size = sizeof(PCIDevice),
    .abstract = true,
    .class_size = sizeof(PCIDeviceClass),
    .class_init = pci_device_class_init,
    .class_base_init = pci_device_class_base_init,
};

static void pci_register_types(void)
{
    type_register_static(&pci_bus_info);
    type_register_static(&pcie_bus_info);
    type_register_static(&cxl_bus_info);
    type_register_static(&conventional_pci_interface_info);
    type_register_static(&cxl_interface_info);
    type_register_static(&pcie_interface_info);
    type_register_static(&pci_device_type_info);
}
```

type_init(pci_register_types)



```
struct DeviceClass {
    /* private: */
    ObjectClass parent_class;

    /* public: */

```

```
struct DeviceState {
    /* private: */
    Object parent_obj;
    /* public: */

```

```
struct PCIDeviceClass {
    DeviceClass parent_class;

```

```
struct PCIDevice {
    DeviceState qdev;

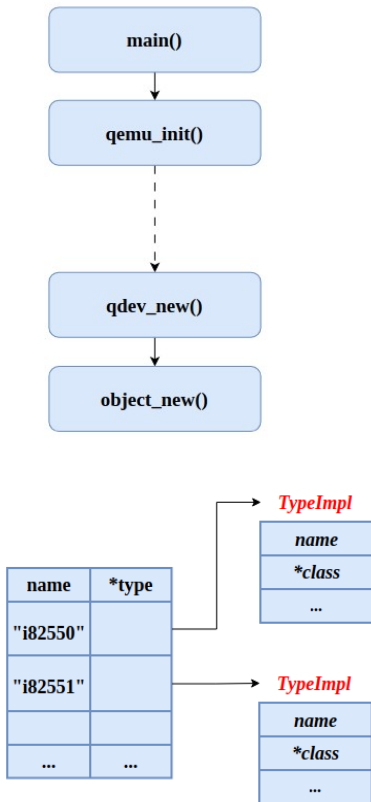
```

```
static Property pci_props[] = {
    DEFINE_PROP_PCI_DEVFN("addr", PCIDevice, devfn, -1),
    DEFINE_PROP_STRING("romfile", PCIDevice, romfile),
    DEFINE_PROP_UINT32("romsize", PCIDevice, romsize, -1),
    DEFINE_PROP_UINT32("rombar", PCIDevice, rom_bar, 1),
    DEFINE_PROP_BIT("multifunction", PCIDevice, cap_present,
        QEMU_PCI_CAP_MULTIFUNCTION_BITNR, false),
    DEFINE_PROP_BIT("x-pcie-lnksta-dlla", PCIDevice, cap_present,
        QEMU_PCIE_LNKSTA_DLLLA_BITNR, true),
    DEFINE_PROP_BIT("x-pcie-extcap-init", PCIDevice, cap_present,
        QEMU_PCIE_EXTCAP_INIT_BITNR, true),
    DEFINE_PROP_STRING("failover_pair_id", PCIDevice,
        failover_pair_id),
    DEFINE_PROP_UINT32("acpi-index", PCIDevice, acpi_index, 0),
    DEFINE_PROP_BIT("x-pcie-err-unc-mask", PCIDevice, cap_present,
        QEMU_PCIE_ERR_UNC_MASK_BITNR, true),
    DEFINE_PROP_BIT("x-pcie-ari-nextfn-1", PCIDevice, cap_present,
        QEMU_PCIE_ARI_NEXTFN_1_BITNR, false),
    DEFINE_PROP_END_OF_LIST();
};
```

QEMU Device Model

Device Type Instantiation & Initialization

- **qdev_new()** creates the devices and initialize the different classes and objects.
- Using **object_new(typename)** a new “instance” will be instantiated from the type.
- **type_get_by_name()** performs a table lookup for our type and returns a **TypeImpl**, we feed it to **object_new_with_type()**.
- Before an “instance” is initialized the “class” must be initialized first through **type_initialize()**, as there should be at least one “class” and may be many “instances”.
- Actually **type_initialize()** does many things



```
Object *object_new(const char *typename)
{
    TypeImpl *ti = type_get_by_name(typename);

    return object_new_with_type(ti);
}

static Object *object_new_with_type(Type type)
{
    Object *obj;
    size_t size, align;
    void (*obj_free)(void *);

    g_assert(type != NULL);
    type_initialize(type);

    size = type->instance_size;
    align = type->instance_align;

    /*
     * Do not use qemu_memalign unless required. Depending on the
     * implementation, extra alignment implies extra overhead.
     */
    if (likely(align <= __alignof__(qemu_max_align_t))) {
        obj = g_malloc(size);
        obj_free = g_free;
    } else {
        obj = qemu_memalign(align, size);
        obj_free = qemu_vfree;
    }

    object_initialize_with_type(obj, size, type);
    obj->free = obj_free;

    return obj;
}
```

QEMU Device Model

Device Type Instantiation & Initialization

- `type_initialize()`
 - First **malloc** the class size for the type if the class exist.
 - Initializing any parent classes.
 - The parent class will be **memcopied** into the current class and any additional memory in the class will be zero filled. The effect of this is that classes automatically inherit any virtual function pointers that the parent class has already initialized.
 - Then all interfaces of the parent class will be initialized first using **type_initialize_interface()**.
 - Then, our type interfaces will be initialized.
 - Finally our `TypeInfo::class_init` is called to let the class being instantiated providing the default initialization for its virtual functions.

```
struct TypeImpl {
    ...
    ObjectClass *class;
    ...
};

static const TypeInfo object_info = {
    .name = TYPE_OBJECT,
    .instance_size = sizeof(Object),
    .class_init = object_class_init,
    .abstract = true,
};

static const TypeInfo device_type_info = {
    .name = TYPE_DEVICE,
    .parent = TYPE_OBJECT,
    .instance_size = sizeof(DeviceState),
    .instance_init = device_initfn,
    .instance_post_init = device_post_init,
    .instance_finalize = device_finalize,
    .class_base_init = device_class_base_init,
    .class_init = device_class_init,
    .abstract = true,
    .class_size = sizeof(DeviceClass),
    .interfaces = (InterfaceInfo[]) {
        { TYPE_VMSTATE_IF },
        { TYPE_RESETTABLE_INTERFACE },
        { }
    }
};

static const TypeInfo pci_device_type_info = {
    .name = TYPE_PCI_DEVICE,
    .parent = TYPE_DEVICE,
    .instance_size = sizeof(PCIDevice),
    .abstract = true,
    .class_size = sizeof(PCIDeviceClass),
    .class_init = pci_device_class_init,
    .class_base_init = pci_device_class_base_init,
};
```

```
static void type_initialize(TypeImpl *ti)
{
    TypeImpl *parent;
    if (ti->class) {
        return;
    }

    ti->class = g_malloc0(ti->class_size);
    parent = type_get_parent(ti);
    if (parent) {
        type_initialize(parent);
        GSList *e;
        int i;
        memcpy(ti->class, parent->class, parent->class_size);
        ti->class->interfaces = NULL;

        for (e = parent->class->interfaces; e; e = e->next) {
            InterfaceClass *iface = e->data;
            ObjectClass *klass = OBJECT_CLASS(iface);

            type_initialize_interface(ti, iface->interface_type, klass->type);
        }

        for (i = 0; i < ti->num_interfaces; i++) {
            TypeImpl *t = type_get_by_name(ti->interfaces[i].typename);
            if (!t) {
                error_report("missing interface '%s' for object '%s'",
                             ti->interfaces[i].typename, parent->name);
            }

            type_initialize_interface(ti, t, t);
        }
    }

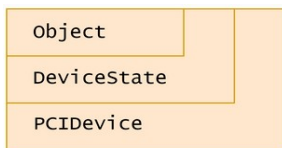
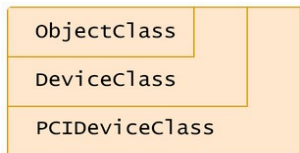
    ti->class->properties = g_hash_table_new_full(g_str_hash, g_str_equal, NULL,
                                                  object_property_free);
    ti->class->type = ti;
    while (parent) {
        if (parent->class_base_init) {
            parent->class_base_init(ti->class, ti->class_data);
        }
        parent = type_get_parent(parent);
    }

    if (ti->class_init) {
        ti->class_init(ti->class, ti->class_data);
    }
}
```

QEMU Device Model

Device Type Instantiation & Initialization

- **object_new_with_type()** after `type_initialize()`, it goes to allocate our object calling **object_initialize_with_type()**.
- **object_initialize_with_type()**
Our instance object should be pointing to the class.
Then initialize the class properties for the obj instance through **object_class_property_init_all()**.
- Then call **object_init_with_type()** that recursively invoke the constructor (**instance_init**) of the parent type.
Finally, we call the instance initialization routine for own object.



```
static void object_class_property_init_all(Object *obj)
{
    ObjectPropertyIterator iter;
    ObjectProperty *prop;

    object_class_property_iter_init(&iter, object_get_class(obj));
    while ((prop = object_property_iter_next(&iter))) {
        if (prop->init) {
            prop->init(obj, prop);
        }
    }
}

static void object_init_with_type(Object *obj, TypeImpl *ti)
{
    if (type_has_parent(ti)) {
        object_init_with_type(obj, type_get_parent(ti));
    }

    if (ti->instance_init) {
        ti->instance_init(obj);
    }
}
```

```
static Object *object_new_with_type(Type type)
{
    Object *obj;
    size_t size, align;
    void (*obj_free)(void *);

    g_assert(type != NULL);
    type_initialize(type);

    size = type->instance_size;
    align = type->instance_align;

    /*
     * Do not use qemu_memalign unless required. Depending on the
     * implementation, extra alignment implies extra overhead.
     */
    if (likely(align <= __alignof__(qemu_max_align_t))) {
        obj = g_malloc(size);
        obj_free = g_free;
    } else {
        obj = qemu_memalign(align, size);
        obj_free = qemu_vfree;
    }

    object_initialize_with_type(obj, size, type);
    obj->free = obj_free;

    return obj;
}

static void object_initialize_with_type(Object *obj, size_t size, TypeImpl *type)
{
    type_initialize(type);

    g_assert(type->instance_size >= sizeof(Object));
    g_assert(type->abstract == false);
    g_assert(size >= type->instance_size);

    memset(obj, 0, type->instance_size);
    obj->class = type->class;
    object_ref(obj);
    object_class_property_init_all(obj);
    obj->properties = g_hash_table_new_full(g_str_hash, g_str_equal,
                                           NULL, object_property_free);

    object_init_with_type(obj, type);
    object_post_init_with_type(obj, type);
}
```

QEMU Device Model

Device Life Cycle

- Instantiation
 - QEMU goes through a series of function calls during the initialization phase.
 - **qdev_new()** creates the devices and initialize the different classes, objects.
 - **device_initfn()** mainly initializes the device properties.
realized : false, indicating that the device has not been realized.
 - **qdev_realize()** checks the device bus type and sets the parent bus if necessary, and finally calls the **object_property_set_bool()** to set the realized attribute indicating that the device has been instantiated and is ready to run.

```
bool qdev_realize(DeviceState *dev, BusState *bus, Error **errp)
{
    assert(!dev->realized && !dev->parent_bus);

    if (bus) {
        if (!qdev_set_parent_bus(dev, bus, errp)) {
            return false;
        }
    } else {
        assert(!DEVICE_GET_CLASS(dev)->bus_type);
    }

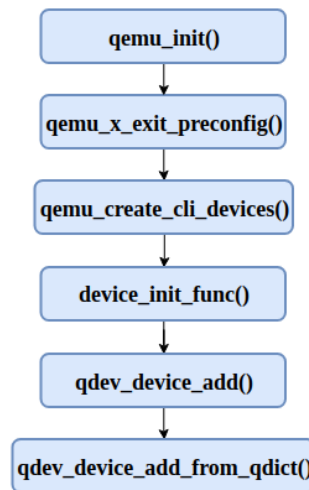
    return object_property_set_bool(OBJECT(dev), "realized", true, errp);
}
```

```
static void device_initfn(Object *obj)
{
    DeviceState *dev = DEVICE(obj);

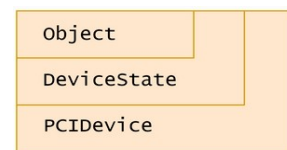
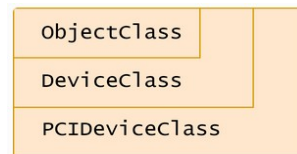
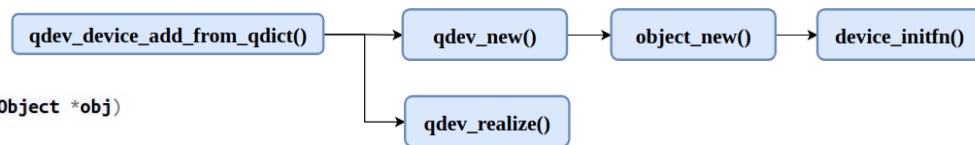
    if (phase_check(PHASE_MACHINE_READY)) {
        dev->hotplugged = 1;
        qdev_hot_added = true;
    }

    dev->instance_id_alias = -1;
    dev->realized = false;
    dev->allow_unplug_during_migration = false;

    QLIST_INIT(&dev->gpios);
    QLIST_INIT(&dev->clocks);
}
```



```
static const TypeInfo device_type_info = {
    .name = TYPE_DEVICE,
    .parent = TYPE_OBJECT,
    .instance_size = sizeof(DeviceState),
    .instance_init = device_initfn,
    .instance_post_init = device_post_init,
    .instance_finalize = device_finalize,
    .class_base_init = device_class_base_init,
    .class_init = device_class_init,
    .abstract = true,
    .class_size = sizeof(DeviceClass),
    .interfaces = (InterfaceInfo[]) {
        { TYPE_VMSTATE_IF },
        { TYPE_RESETTABLE_INTERFACE },
    }
};
```



QEMU Device Model

Device Life Cycle

- Destruction
 - It goes to remove all what's added for the device.
 - Device → “unrealized”.
 - Remove the device from bus.
 - **device_finalize()**.
 - Destroy the device object.
 - Unregister the device.

```
static const TypeInfo device_type_info = {
    .name = TYPE_DEVICE,
    .parent = TYPE_OBJECT,
    .instance_size = sizeof(DeviceState),
    .instance_init = device_initfn,
    .instance_post_init = device_post_init,
    .instance_finalize = device_finalize,
    .class_base_init = device_class_base_init,
    .class_init = device_class_init,
    .abstract = true,
    .class_size = sizeof(DeviceClass),
    .interfaces = (InterfaceInfo[]) {
        { TYPE_VMSTATE_IF },
        { TYPE_RESETTABLE_INTERFACE },
        { }
    }
};
```

PC Hardware Initialization

Machine Initialization

- Generally QEMU is executed with passing command line arguments **argv**.
- Ex.
qemu-system-x86_64 -kernel kernel/linux/arch/x86_64/boot/bzImage -initrd build/build/initramfs-busybox-x86.cpio.gz -device pci-dev-emu,bus=pci.0 -nographic -append "console=ttyS0" -enable-kvm
- QEMU creates a list for the machine options **machine_opts_dict**.
- **select_machine()** starts by calling **object_class_get_list()**. which will give us all object classes of type **machine**.
- A hardware machine is represented with the **MachineClass** class, and an instance is represented with **MachineState**.

```
void qemu_init(int argc, char **argv)
{
    ...
    qemu_create_machine(machine_opts_dict);
    ...
}
```

```
static void qemu_create_machine(QDict *qdict)
{
    MachineClass *machine_class = select_machine(qdict, &error_fatal);
    ...
}

static MachineClass *select_machine(QDict *qdict, Error **errp)
{
    const char *optarg = qdict_get_try_str(qdict, "type");
    GSList *machines = object_class_get_list(TYPE_MACHINE, false);
    ...
    machine_class = find_machine(optarg, machines);
}
```

```
struct MachineClass {
    /*< private >*/
    ObjectClass parent_class;
    /*< public >*/
```

```
struct MachineState {
    /*< private >*/
    Object parent_obj;
    /*< public >*/
```

PC Hardware Initialization

Machine Initialization

- By using **DEFINE_PC_MACHINE()** we can register a new machine type.

- Ex.
I440FX Intel machine

```
static void pc_init1(MachineState *machine,
                    const char *host_type, const char *pci_type)
{
    PCMachineState *pcms = PC_MACHINE(machine);
    PCMachineClass *pcmc = PC_MACHINE_GET_CLASS(pcms);
    X86MachineState *x86ms = X86_MACHINE(machine);
    MemoryRegion *system_memory = get_system_memory();
    MemoryRegion *system_io = get_system_io();
    ...
    x86_cpus_init(x86ms, pcmc->default_cpu_version);
    ...
}
```

```
struct MachineClass {
    /*< private >*/
    ObjectClass parent_class;
    /*< public >*/
```

```
struct X86MachineClass {
    /*< private >*/
    MachineClass parent;
```

```
struct PCMachineClass {
    /*< private >*/
    X86MachineClass parent_class;
```

```
struct MachineState {
    /*< private >*/
    Object parent_obj;
    /*< public >*/
```

```
struct X86MachineState {
    /*< private >*/
    MachineState parent;
```

```
typedef struct PCMachineState {
    /*< private >*/
    X86MachineState parent_obj;
```

```
DEFINE_I440FX_MACHINE(v9_1, "pc-i440fx-9.1", NULL,
                    pc_i440fx_9_1_machine_options);
```

```
#define DEFINE_I440FX_MACHINE(suffix, name, compatfn, optionfn) \
{ \
    static void pc_init_##suffix(MachineState *machine) \
    { \
        void (*compat)(MachineState *m) = (compatfn); \
        if (compat) { \
            compat(machine); \
        } \
        pc_init1(machine, TYPE_I440FX_PCI_HOST_BRIDGE, \
            TYPE_I440FX_PCI_DEVICE); \
    } \
    DEFINE_PC_MACHINE(suffix, name, pc_init_##suffix, optionfn)
```

```
#define DEFINE_PC_MACHINE(suffix, namestr, initfn, optsfn) \
static void pc_machine_##suffix##_class_init(ObjectClass *oc, void *data) \
{ \
    MachineClass *mc = MACHINE_CLASS(oc); \
    optsfn(mc); \
    mc->init = initfn; \
    mc->kvm_type = pc_machine_kvm_type; \
} \
static const TypeInfo pc_machine_type_##suffix = { \
    .name = namestr TYPE_MACHINE_SUFFIX, \
    .parent = TYPE_PC_MACHINE, \
    .class_init = pc_machine_##suffix##_class_init, \
}; \
static void pc_machine_init_##suffix(void) \
{ \
    type_register(&pc_machine_type_##suffix); \
} \
type_init(pc_machine_init_##suffix)
```

PC Hardware Initialization

Virtual CPUs

- **x86_cpus_init()** starts by creating the CPU topology structure (number of threads, sockets, cores, ..).
- Loop through the number of vCPUs to call **x86_cpu_new()** that calls **object_new()** which creates the new CPU objects.
- **qdev_realize()** then sets the property device to *realized*.

```
static void pc_init1(MachineState *machine,
                    const char *host_type, const char *pci_type)
{
    PCMachineState *pcms = PC_MACHINE(machine);
    PCMachineClass *pcmc = PC_MACHINE_GET_CLASS(pcms);
    X86MachineState *x86ms = X86_MACHINE(machine);
    MemoryRegion *system_memory = get_system_memory();
    MemoryRegion *system_io = get_system_io();
    ...
    x86_cpus_init(x86ms, pcmc->default_cpu_version);
    ...
}

void x86_cpus_init(X86MachineState *x86ms, int default_cpu_version)
{
    ...
    possible_cpus = mc->possible_cpu_arch_ids(ms);
    for (i = 0; i < ms->smp.cpus; i++) {
        x86_cpu_new(x86ms, possible_cpus->cpus[i].arch_id, &error_fatal);
    }
}

void x86_cpu_new(X86MachineState *x86ms, int64_t apic_id, Error **errp)
{
    Object *cpu = object_new(MACHINE(x86ms)->cpu_type);

    if (!object_property_set_uint(cpu, "apic-id", apic_id, errp)) {
        goto out;
    }
    qdev_realize(DEVICE(cpu), NULL, errp);

out:
    object_unref(cpu);
}
```

Virtual Memory (SoftMMU)

DIMM

- QEMU emulates the memories as a HW component/device installed in the system.
- QEMU emulates the **DIMM** (Dual Inline Memory Module) that represents the physical part that the RAM is in.
- A DIMM (Dual Inline Memory Module) is a small circuit board that holds memory chips, and is also known as a RAM stick.
- The amount of RAM available on a DIMM depends on the number of memory chips on it and the individual capacity of each chip, so one DIMM may have more memory than another by having more memory chips, higher-capacity memory chips.
- QEMU uses the same mechanism found in real hardware, it emulates a DIMM hotplug so the guest OS can detect that a new memory had been added or removed from a memory slot.
- DIMM is represented by **PCDIMMDevice** structure.
- The guest RAM itself is not contained within to pc-dimm object, but it is associated with a **HostMemoryBackend** object .
- **HostMemoryBackend** is mainly defined by a **MemoryRegion** structure.



```
struct MemoryRegion {
    Object parent_obj;

    /* private: */

    /* The following fields should fit in a cache line */
    bool romd_mode;
    bool ram;
    bool subpage;
    bool readonly; /* For RAM regions */
    bool nonvolatile;
    bool rom_device;
    bool flush_coalesced_mmtio;
    uint8_t dirty_log_mask;
    bool is_ionmu;
    RAMBlock *ram_block;
    Object *owner;
```

```
struct PCDIMMDevice {
    /* private */
    DeviceState parent_obj;

    /* public */
    uint64_t addr;
    uint32_t node;
    int32_t slot;
    HostMemoryBackend *hostmem;
};

struct HostMemoryBackend {
    /* private */
    Object parent;

    /* protected */
    uint64_t size;
    bool merge, dump, use_canonical_path;
    bool prealloc, is_mapped, share, reserve;
    uint32_t prealloc_threads;
    ThreadContext *prealloc_context;
    DECLARE_BITMAP(host_nodes, MAX_NODES + 1);
    HostMemPolicy policy;

    MemoryRegion mr;
};
```

Virtual Memory (SoftMMU)

Key Structures

- Generally the guest RAM can be backed by **anonymous memory** or a **file**.
- There are also other file backend options like **shmfs** RAM filesystems or persistent memory (**pmem**) backend.
- Ex. Host memory → file representation
It ends to allocate memory from a created file and memmapped the memory different chunks to be used.

```
static void
file_backend_class_init(ObjectClass *oc, void *data)
{
    HostMemoryBackendClass *bc = MEMORY_BACKEND_CLASS(oc);

    bc->alloc = file_backend_memory_alloc;
    ...
}

static void
file_backend_memory_alloc(HostMemoryBackend *backend, Error **errp)
{
    ...
    memory_region_init_ram_from_file(&backend->mr, OBJECT(backend), name,
                                     backend->size, fb->align, ram_flags,
                                     fb->mem_path, fb->readonly, errp);
    ...
}
#endif
```

```
struct HostMemoryBackend {
    /* private */
    Object parent;

    /* protected */
    uint64_t size;
    bool merge, dump, use_canonical_path;
    bool prealloc, is_mapped, share, reserve;
    uint32_t prealloc_threads;
    ThreadContext *prealloc_context;
    DECLARE_BITMAP(host_nodes, MAX_NODES + 1);
    HostMemPolicy policy;

    MemoryRegion mr;
};

static const TypeInfo file_backend_info = {
    .name = TYPE_MEMORY_BACKEND_FILE,
    .parent = TYPE_MEMORY_BACKEND,
    .class_init = file_backend_class_init,
    .instance_finalize = file_backend_instance_finalize,
    .instance_size = sizeof(HostMemoryBackendFile),
};

static void register_types(void)
{
    type_register_static(&file_backend_info);
}

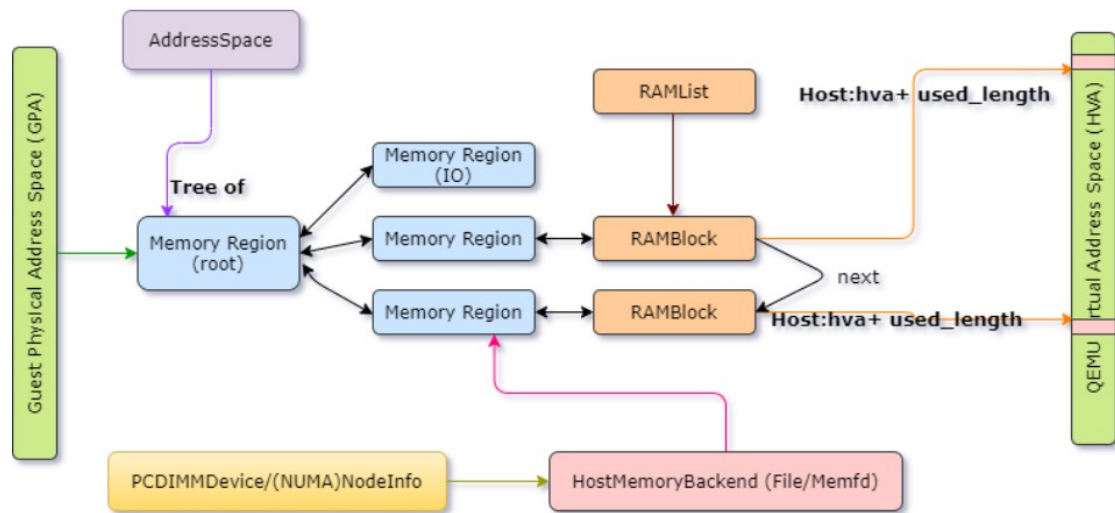
type_init(register_types);
```

Virtual Memory (SoftMMU)

Key Structures

- A tree of memory regions forms an address space, it is represented by the **AddressSpace** structure which describes a mapping of addresses to **MemoryRegion** objects.
- Those memory regions objects don't carry the guest memory directly, instead they are associated with RAMBlocks. It is the **RAMBlock** which represents a single malloc to a mmaped chunk of memory.

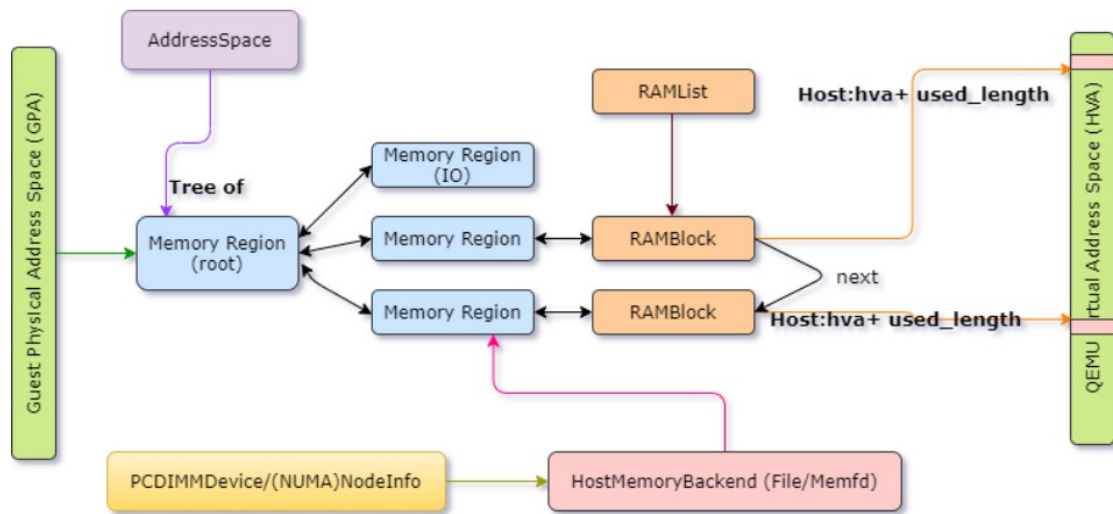
```
struct AddressSpace {  
    /* private: */  
    struct rcu_head rcu;  
    char *name;  
    MemoryRegion *root;  
  
    /* Accessed via RCU. */  
    struct FlatView *current_map;  
  
    int ioeventfd_nb;  
    struct MemoryRegionIoeventfd *ioeventfds;  
    QTAILQ_HEAD(, MemoryListener) listeners;  
    QTAILQ_ENTRY(AddressSpace) address_spaces_link;  
};
```



Virtual Memory (SoftMMU)

Key Structures

- Full Picture

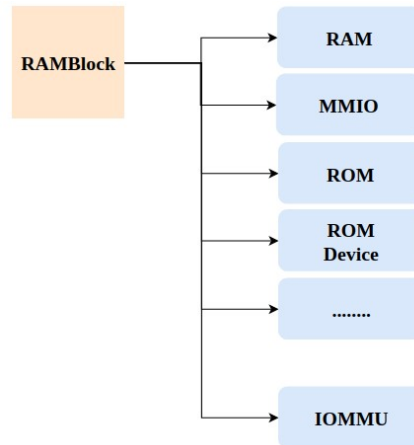


Virtual Memory (SoftMMU)

Key Structures

- RAMBlock types

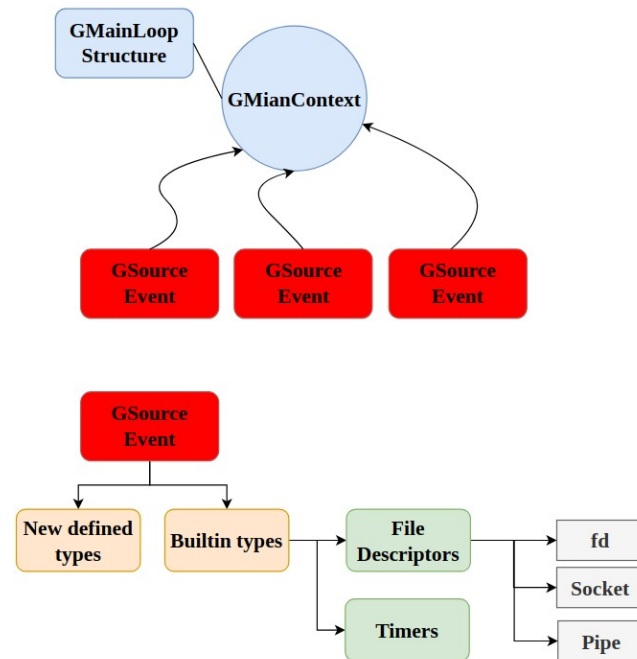
```
struct RAMBlock {
    struct rcu_head rcu;
    struct MemoryRegion *mr;
    uint8_t *host;
    uint8_t *colo_cache;
    ram_addr_t offset;
    ram_addr_t used_length;
    ram_addr_t max_length;
    void (*resized)(const char*, uint64_t length, void *host);
    uint32_t flags;
    char idstr[256];
    QLIST_ENTRY(RAMBlock) next;
    QLIST_HEAD(, RAMBlockNotifier) ramblock_notifiers;
    int fd;
    size_t page_size;
    unsigned long *bmap;
    unsigned long *receivedmap;
    unsigned long *clear_bmap;
    uint8_t clear_bmap_shift;
    ram_addr_t postcopy_length;
};
```



QEMU Architecture

Event Loop Mechanism

- QEMU adopts an event-driven architecture.
- The event loop mechanism is implemented based on **glib**.
 - glib is a cross-platform collection of low-level libraries written in C.
 - glib uses **GMainLoop** structure to represent a main event loop.
 - After adding the initial event sources, and then running the created main event loop, where the **GMainContext** is used to represent its context and encapsulates the state and behavior of the event loop.
 - Event sources are represented by **GSource**.
 - Each **GSource** can be associated with multiple file descriptors.
 - In order to be able to handle multiple independent event sources in different threads, each **GSource** will be associated with one **GMainContext**.
 - **GMainContext** can only run in a single thread, but event sources can be added or deleted by other threads.

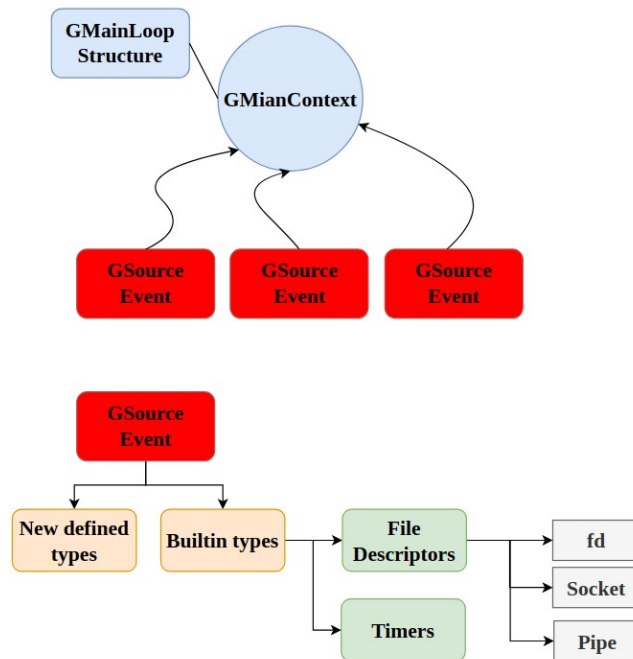


QEMU Architecture

Event Loop Mechanism

- QEMU's event loop mechanism is one of its core functions, which is used to coordinate various asynchronous operations of the virtual machine.
- By monitoring and scheduling various event sources, it allows QEMU to effectively respond to external inputs, timer events, etc., thereby achieving high-performance virtualization.
- QEMU customizes a new event source **AioContext** which inherits from **GSource** structure provided by glib.

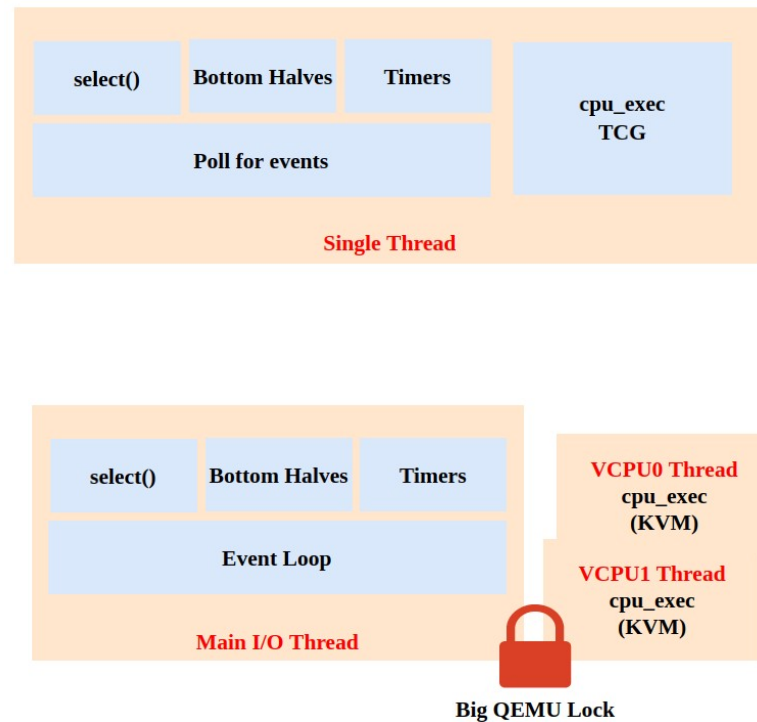
```
struct AioContext {  
    GSource source;  
};
```



QEMU Architecture

Architecture

- V0.15
 - The cpu is executing guest code, and every 1ms, QEMU will poll for events, when a file descriptor (fd) becomes ready via the **select()** syscall, a timer expires, or a BH (Bottom halves).
 - Each event raised QEMU will invoke a callback that responds to that event.
- V1.0
 - There is a dedicated thread to handle IO events, which is called the I/O thread.
 - This thread runs an event loop that poll for events and process them.
 - Executing guest code became multithreaded when used in conjunction with KVM, but not with TCG.
 - The synchronization between the thread that runs guest code and the **I/O thread** is done with a lock that is called the big QEMU lock (BQL).



QEMU Architecture

Architecture

- Now
 - The previous architecture approaches only one single thread will be multiplexing between the four vCPUs and the event loop, resulting in a poor performance for symmetric multiprocessing guests.
 - To truly offer support for SMP, additional event loop threads can be created. So basically, we can create multiple event loop threads to spread work across several IO threads instead of just the main event loop.

